STANDARD OPERATING PROCEDURE (SOP) | UNIVERSAL ARCHITECTURE BLUEPRINT

Headless WordPress + Astro Marketing Site

Evergreen Multi-Site Architecture, Engineering & Replication Guide

Doc ID: SOP-ENG-UNIVERSAL | Blueprint: Decoupled High-Growth Sites | Stack: WPGraphQL + Gravity Forms + Astro + React + Vercel

Table of Contents

1. Executive Summary & System Architecture
2. Project Configuration & Placeholder Matrix
3. Phase 1: Headless WordPress Setup, Plugins & PHP Snippets
4. Phase 2: Astro Project Initialization & Configuration
5. Phase 3: Folder Structure & Data-Driven Content Architecture
6. Phase 4: Dual-Mode WordPress API Layer & Security Sanitization
7. Phase 5: Component Splits, Master Templates & Design Rules
8. Phase 6: Gravity Forms React Island (LeadForm.jsx)
9. Phase 7: Vercel Deployment, DNS Cutover & Rebuild Webhooks
10. Phase 8: New Site Replication & Adaptability Checklist

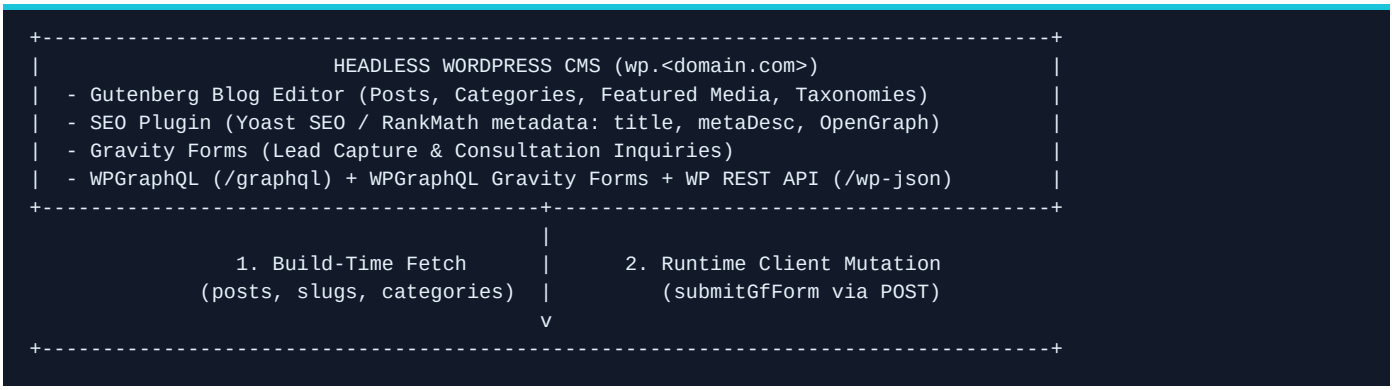
1. Executive Summary & System Architecture

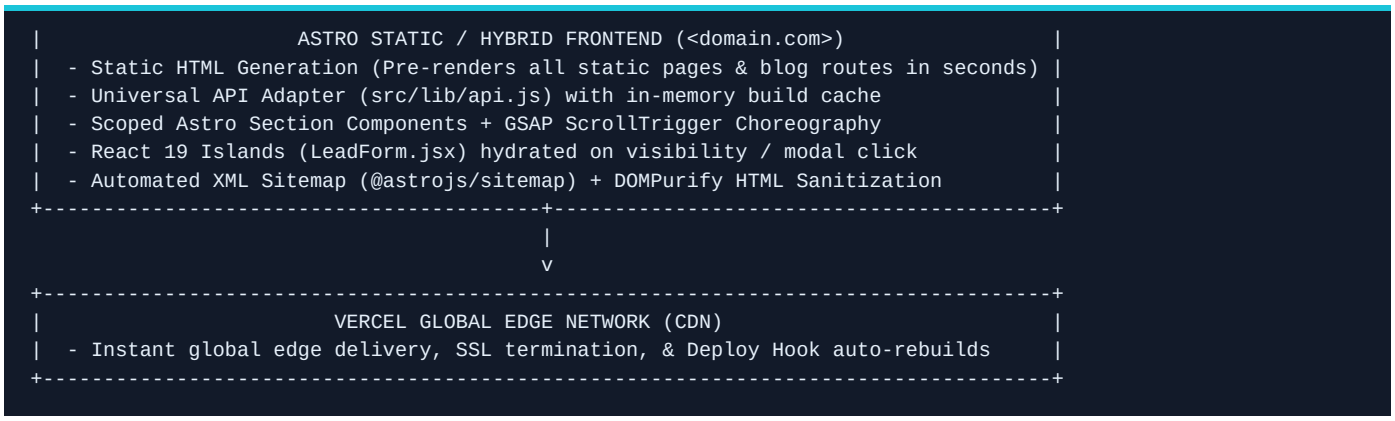
1.1 Purpose & Scope

This Standard Operating Procedure (SOP) provides an evergreen, production-hardened engineering blueprint to build, migrate, and deploy modern marketing websites. By pairing Astro on the frontend with Headless WordPress on the backend, this architecture achieves:

- Sub-second Core Web Vitals (CWV): 100% pre-rendered static HTML via Astro static site generation (SSG).
- Familiar Content Editing: Non-technical marketing teams continue editing blog posts and review form entries inside the standard WordPress admin interface.
- Enterprise Security: WordPress is isolated behind an origin subdomain (wp.<domain.com>), shielded from public web attacks, while the apex domain (<domain.com>) is served entirely via immutable edge cache.
- High-Converting Lead Generation: Interactive React 19 form islands mutate Gravity Forms directly over GraphQL with client-side validation, phone auto-masking, and instant feedback.

1.2 End-to-End Architecture Topology





1.3 Core Technology Stack & Version Matrix

Layer	Package / Tool	Recommended Version	Architectural Purpose
Runtime	Node.js	>= 22.12.0	Modern LTS build environment & ES module execution
Framework	astro	^5.x or ^7.x	Static site generation (SSG), file-based routing, scoped CSS
UI Islands	react / react-dom	^19.x	Interactive client-side lead capture forms (@astrojs/react)
Animation	gsap	^3.15.x	ScrollTrigger parallax, counter animations, & floating graphics
Security	isomorphic-dompurify	^3.x	XSS sanitization for WordPress CMS content & confirmation messages
SEO	@astrojs/sitemap	^3.x	Automated sitemap-index.xml & sitemap-0.xml generation
Icons	material-symbols	^0.40.x	Google Material Symbols Outlined font & ligature system
CMS Backend	WordPress + PHP	6.x / PHP 8.2+	Headless content management, taxonomies & form handling

2. Project Configuration & Placeholder Matrix

When replicating this architecture for a new client, brand, or agency project, replace the following template variables throughout your repositories and server configs:

Variable Token	Description	Example Target Value
<domain.com>	Production apex website domain	acmemarketing.com
<wp.domain.com>	Headless WordPress origin CMS domain	wp.acmemarketing.com
<brand>	Brand or client identifier (kebab-case)	acme-marketing
<brand-frontend>	Frontend Git repository and directory name	acme-marketing-frontend
<wp-server-ip>	IPv4 address of the WordPress hosting server	192.0.2.45
<staging-url>	Staging or preview environment URL	https://staging.acmemarketing.com
<form-id>	Target Gravity Form database ID	1 (or custom ID like 15)

<company-phone>	Master client phone number	(555) 019-2834
<company-email>	Master client contact or notifications email	hello@acmemarketing.com
<PREFIX>_VERCEL_DEPLOY_HOOK	PHP wp-config constant for deployment trigger	ACME_VERCEL_DEPLOY_HOOK

3. Phase 1: Headless WordPress Setup, Plugins & PHP Snippets

3.1 Subdomain Strategy

When decoupling an existing or new WordPress website:

1. Migration / Development Phase: Maintain WordPress on a staging server or the current domain, connecting Astro via PUBLIC_WORDPRESS_API_URL.
2. Production Cutover Phase: Move the WordPress installation to a dedicated origin subdomain (https://<wp.domain.com>) pointed directly to the host server IP (<wp-server-ip>). The apex domain (https://<domain.com>) and www are assigned to Vercel.

3.2 Required WordPress Plugins

Install and activate the following plugins in /wp-admin/plugins.php:

1. WPGraphQL (wp-graphql)
   - Exposes the performant /graphql endpoint for querying posts, categories, media attachments, and authors.
2. Gravity Forms (gravityforms)
   - Commercial form manager for lead capture, CRM webhooks, and email notifications.
3. WPGraphQL for Gravity Forms (wp-graphql-gravity-forms)
   - Provides the submitGfForm mutation enabling client-side submissions directly from React islands.
4. Yoast SEO (wordpress-seo) or RankMath + WPGraphQL SEO Addon
   - Exposes seo { title metaDesc } fields to GraphQL post queries and REST API endpoints.
5. WP Webhooks (wp-webhooks) *(or custom save_post hook)*
   - Automatically triggers the Vercel Deploy Hook when content is published or revised.

3.3 Must-Use Plugin: wp-content/mu-plugins/headless-cms-config.php

Create a Must-Use (MU) plugin in WordPress. Unlike standard theme code, MU plugins are automatically active, cannot be accidentally disabled by admin users, and execute on every request:

```php
<?php
/**
 * Plugin Name: Universal Headless WordPress Configuration
 * Description: Optimizes WPGraphQL query limits, CORS headers, and Vercel build webhooks.
 * Version: 2.0.0
 * Author: Engineering Team
 */

// 1. Increase WPGraphQL max query limit for Astro single-fetch static builds
add_filter( 'graphql_connection_max_query_amount', function( $amount, $source, $args, $context, $info ) {
    return 500;
}, 10, 5 );

// 2. Allow CORS on WPGraphQL endpoint for frontend form submissions & local dev
add_filter( 'graphql_response_headers_to_send', function( $headers ) {
    $allowed_origins = [
        'https://<domain.com>',
```

```
        'https://www.<domain.com>',
        'https://<staging-url>',
        'http://localhost:4321',
        'http://localhost:3000',
    ];

    $origin = isset( $_SERVER['HTTP_ORIGIN'] ) ? $_SERVER['HTTP_ORIGIN'] : '';
    if ( in_array( $origin, $allowed_origins, true ) ) {
        $headers['Access-Control-Allow-Origin']      = $origin;
        $headers['Access-Control-Allow-Methods']     = 'POST, GET, OPTIONS';
        $headers['Access-Control-Allow-Headers']     = 'Content-Type, Authorization';
        $headers['Access-Control-Allow-Credentials'] = 'true';
    }
    return $headers;
} );

// 3. Trigger Vercel Deploy Hook when a post is published or updated
add_action( 'save_post_post', function( $post_id, $post, $update ) {
    if ( wp_is_post_revision( $post_id ) || wp_is_post_autosave( $post_id ) ) {
        return;
    }
    if ( $post->post_status !== 'publish' ) {
        return;
    }

    $deploy_hook = defined( '<PREFIX>_VERCEL_DEPLOY_HOOK' ) ? constant( '<PREFIX>_VERCEL_DEPLOY_HOOK' ) : '';
    if ( ! empty( $deploy_hook ) ) {
        wp_remote_post( $deploy_hook, [
            'timeout'  => 5,
            'blocking' => false,
            'headers'  => [ 'Content-Type' => 'application/json' ],
            'body'     => json_encode( [
                'event'   => 'post_updated',
                'post_id' => $post_id,
                'slug'    => $post->post_name,
            ] ),
        ] );
    }
}, 10, 3 );
```

3.4 Web Application Firewall (WAF) Whitelisting

If Wordfence, Cloudflare WAF, or ModSecurity is installed on the WordPress origin server:

1. Navigate to your security plugin's whitelisted URL options.
2. Whitelist /graphql for HTTP POST requests so client-side submitGfForm submissions from the browser are never blocked with 403 Forbidden.
3. Under WPGraphQL -> Settings, ensure public query execution is enabled for published posts and Gravity Forms mutations.

3.5 Gravity Forms Field Mapping Table

In Forms -> New Form, create the Consultation / Contact Form (e.g., Form ID <form-id>) and match field IDs to GF_FIELD_IDS in the React frontend:

Form Field Label	Field Type	Standard Field ID	GraphQL Mutation Input Key	Required
First Name	Single Line Text	12	value: String	Yes
Last Name	Single Line Text	13	value: String	Yes

Email Address	Email	5	emailValues: { value: String }	Yes
Phone Number	Phone (Standard)	6	value: String	Yes
Company / Organization	Single Line Text	4	value: String	No
Services of Interest	Text / Checkbox	7	value: String (comma-separated)	Yes
Project Details / Message	Paragraph Text	9	value: String	No

4. Phase 2: Astro Project Initialization & Configuration

4.1 Bootstrap Command Sequence

Execute the following commands to create the frontend project repository:

```bash
# 1. Create a minimal Astro project
npm create astro@latest <brand-frontend> -- --template minimal
cd <brand-frontend>

# 2. Install official integrations and core libraries
npm install @astrojs/react @astrojs/sitemap react react-dom gsap isomorphic-dompurify material-symbols

# 3. Install React TypeScript typings
npm install -D @types/react @types/react-dom
```

4.2 Universal package.json Specification

```json
{
  "name": "<brand-frontend>",
  "type": "module",
  "version": "1.0.0",
  "engines": {
    "node": ">=22.12.0"
  },
  "scripts": {
    "dev": "astro dev",
    "build": "astro build",
    "preview": "astro preview",
    "astro": "astro"
  },
  "dependencies": {
    "@astrojs/react": "^7.0.1",
    "@astrojs/sitemap": "^3.7.4",
    "@types/react": "^19.3.0",
    "@types/react-dom": "^19.3.0",
    "astro": "^7.3.8",
    "gsap": "^3.15.0",
    "isomorphic-dompurify": "^3.19.0",
    "material-symbols": "^0.40.2",
    "react": "^19.3.1",
    "react-dom": "^19.3.1"
  }
}
```

4.3 Universal astro.config.mjs

```javascript
import { defineConfig } from 'astro/config';
import react from '@astrojs/react';
import sitemap from '@astrojs/sitemap';

export default defineConfig({
  site: process.env.SITE_URL || 'https://<domain.com>',
  base: '/',
  integrations: [
    react(),
    sitemap({
      filter: (page) => !page.includes('/landing/thank-you'),
    }),
  ],
});
```

4.4 Environment Configuration (.env and .env.example)

Create a .env file in the project root:

```ini
# Production WordPress GraphQL Endpoint
PUBLIC_WORDPRESS_API_URL=https://<wp.domain.com>/graphql
PUBLIC_WORDPRESS_GRAPHQL_URL=https://<wp.domain.com>/graphql

# Optional REST API Fallback (for staging before GraphQL setup)
# PUBLIC_WORDPRESS_API_URL=https://<wp.domain.com>/wp-json/wp/v2

# Canonical Production Site URL
SITE_URL=https://<domain.com>
```

5. Phase 3: Folder Structure & Data-Driven Content Architecture

5.1 Standard Project Directory Tree

```text
<brand-frontend>/
+-- .env                            # Local environment variables
+-- .env.example                    # Template environment variables
+-- astro.config.mjs                # Astro configuration (React, Sitemap, Canonical Site URL)
+-- package.json                    # Dependencies, engine version & npm scripts
+-- public/                         # Static public assets (served at root "/")
|   +-- favicon.svg                 # Primary vector brand favicon
|   +-- fonts/                      # Self-hosted typography (WOFF2)
|   `-- assets/                     # Categorized visual design assets
|       +-- main-graphics/          # 3D Centerpiece illustrations (1 per stage)
|       +-- decorations/            # Background blobs, SVG grids & orbital satellites
|       +-- icons/                  # Custom SVG iconography & badges
|       +-- case-studies/           # Portfolio client photography & project screenshots
|       `-- landing-pages/          # High-conversion industry hero illustrations
`-- src/
    +-- config/
    |   `-- site.js                 # Global single source of truth (contact, phone, links)
    +-- styles/
    |   +-- global.css              # CSS custom properties, buttons, reset, typography
    |   `-- gutenberg.css           # Scoped styles for WordPress HTML blog articles
    +-- lib/
    |   +-- api.js                  # Dual-mode REST/GraphQL client with single-fetch cache
```

```
|   `-- animations.js            # GSAP ScrollTrigger orchestration & floating tweens
+-- layouts/
|   +-- MainLayout.astro         # Universal header, navigation mega-menu, footer, lead modal
|   `-- LandingLayout.astro      # Focused conversion layout for dedicated marketing campaigns
+-- components/
|   +-- BaseHead.astro           # Meta tags, OpenGraph, Twitter cards, canonical tags
|   +-- ServiceHero.astro        # Reusable hero banner for service pages
|   +-- StartsSection.astro      # 2-column spotlight feature + animated SVG stat ring
|   +-- StatsBand.astro          # High-impact dark KPI number counter strip
|   +-- ProcessSection.astro     # Interactive 4-step accordion + synced illustration stage
|   +-- PillarsSection.astro     # Numbered strategic pillars with parallax accents
|   +-- DeliverablesGrid.astro   # 3-column card grid with floating decorative badges
|   +-- ReadyBand.astro          # Dark CTA surface extending behind consult wave ribbon
|   +-- ConsultSection.astro     # Dual-mode wave ribbon + LeadForm integration
|   +-- CaseStudyHero.astro      # Case study showcase hero banner
|   +-- CaseStudyNarrative.astro  # Problem, Solution, Results structured case study layout
|   +-- LightboxModal.astro      # Fullscreen accessible image gallery viewer
|   +-- LeadForm.jsx             # React 19 Gravity Forms client island
|   `-- LeadForm.css             # Form layout, validation error alerts, pill selectors
+-- data/
|   +-- homeData.js              # Homepage hero slides, process steps, service previews
|   +-- services/                # Modular service data models (1 file per service)
|   |   +-- webDesignData.js
|   |   +-- seoData.js
|   |   `-- *.js
|   +-- landing/                 # Industry-specific PPC campaign models
|   `-- work/                    # Case studies registry & individual client project files
`-- pages/
    +-- index.astro              # Homepage
    +-- about.astro              # Company narrative & team overview
    +-- services.astro           # Services directory index
    +-- services/
    |   `-- [service].astro      # Dynamic or static service detail page templates
    +-- work.astro               # Portfolio case study directory
    +-- work/[slug].astro        # Canonical case study dynamic route
    `-- blog/
        +-- index.astro          # Blog archive with category pill filtering
        `-- [slug].astro         # Dynamic pre-rendered WordPress blog article
```

5.2 Zero Inline CSS Data Architecture

A foundational rule of this architecture is 100% separation between pure data and presentation coordinates:

- Data Files (src/data//*.js): Must NEVER contain inline styles (e.g. style: "width: 20%; right: 6%;"). Instead, data models specify only content text, media paths, parallax velocity numbers, and semantic Directional Placement** tokens:

```javascript
placement: "top-left" | "top-right" | "bottom-left" | "bottom-right"
```

- Astro Components (src/components/*.astro): Own all layout rules, responsive percentages, z-indexes, and default intrinsic dimensions (width={sat.width ?? 746}, height={sat.height ?? 780}).
- Why this is critical: GSAP transforms (translate3d, rotate) are written dynamically to the HTML style attribute. Keeping initial styles inside scoped CSS classes prevents GSAP's clearProps from erasing layout coordinates when viewport size changes.

Example Clean Data Model (src/data/services/sampleServiceData.js)

```javascript
export const startsData = {
  eyebrow: "GROWTH-DRIVEN ARCHITECTURE",
  title: "Engineering Websites That Convert Visitors Into Revenue",
  question: "Is your digital presence driving measurable pipeline, or sitting idle?",
  body: [
    "A high-performing marketing website is not a digital brochure; it is your primary revenue engine...",
    "We pair lightning-fast static page generation with conversion-engineered design.",
  ],
  callout: {
    percent: 85,
    title: "MOBILE CONVERSION BENCHMARK",
    text: "Over <strong>85% of target buyers</strong> evaluate B2B and consumer brands on a mobile device.",
  },
  media: {
    blob: {
      src: "/assets/decorations/circular-glow-backdrop.svg",
    },
    main: {
      src: "/assets/main-graphics/responsive-device-showcase.svg",
      alt: "Multi-device responsive application showcase",
    },
    satellites: [
      {
        src: "/assets/decorations/performance-badge.svg",
        parallax: -35,
        placement: "top-left",
      },
      {
        src: "/assets/decorations/conversion-metric-bubble.svg",
        parallax: -45,
        placement: "bottom-right",
      },
    ],
    pill: {
      icon: "speed",
      title: "100/100 PageSpeed",
      subtitle: "Instant sub-second edge delivery",
      parallax: -25,
    },
  },
};
```

6. Phase 4: Dual-Mode WordPress API Layer & Security Sanitization

6.1 Universal API Adapter (src/lib/api.js)

src/lib/api.js automatically adapts between GraphQL and REST based on PUBLIC_WORDPRESS_API_URL:

- If the URL ends with /graphql, queries execute via GraphQL POST.
- If pointing to /wp-json, it transparently switches to REST and normalizes the payload into the exact same unified schema.
- In-Memory Build Cache: During astro build, all published articles are retrieved in a single batched query (posts(first: 500)). Generating hundreds of static routes executes with minimal network requests, cutting build times from minutes down to ~12 seconds.

JAVASCRIPT

```
// src/lib/api.js
const API_URL = (
  (typeof import.meta !== "undefined" && import.meta.env?.PUBLIC_WORDPRESS_API_URL) ||
  (typeof process !== "undefined" && process.env?.PUBLIC_WORDPRESS_API_URL) ||
  "https://<wp.domain.com>/graphql"
).replace(/\/$/, "");

const isGraphQL = API_URL.endsWith("/graphql");

// In-memory static build cache
let cachedPosts = null;
let cachedPostBySlug = new Map();

export async function ensureFullPostsCache() {
  if (cachedPosts) return cachedPosts;

  if (isGraphQL) {
    const query = `
      query GetAllPostsForStaticBuild {
        posts(first: 500, where: { status: PUBLISH }) {
          nodes {
            id
            databaseId
            title
            slug
            date
            excerpt
            content
            featuredImage {
              node { sourceUrl altText }
            }
            categories {
              nodes { databaseId name slug }
            }
            seo { title metaDesc }
          }
        }
      }
    `;
    const res = await fetch(API_URL, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ query }),
    });
    const json = await res.json();
    cachedPosts = json.data?.posts?.nodes || [];
  } else {
    // REST API fallback with pagination
    const res = await fetch(`${API_URL}/posts?per_page=100&_embed=1`);
    const raw = await res.json();
    cachedPosts = raw.map(normalizeRestPost);
  }

  for (const post of cachedPosts) {
    cachedPostBySlug.set(post.slug, post);
  }
  return cachedPosts;
}

export async function getAllPostSlugs() {
```

```
  const posts = await ensureFullPostsCache();
  return posts.map((p) => ({ slug: p.slug }));
}

export async function getPostBySlug(slug) {
  await ensureFullPostsCache();
  return cachedPostBySlug.get(slug) || null;
}
```

6.2 XSS Sanitization (isomorphic-dompurify)

Never render raw CMS HTML in templates. Always clean inbound post content with DOMPurify before injecting it into the DOM:

ASTRO

```
---
// src/pages/blog/[slug].astro
import DOMPurify from "isomorphic-dompurify";
import MainLayout from "../../layouts/MainLayout.astro";
import { getAllPostSlugs, getPostBySlug } from "../../lib/api.js";

export async function getStaticPaths() {
  const slugs = await getAllPostSlugs();
  return slugs.map(({ slug }) => ({ params: { slug } }));
}

const { slug } = Astro.params;
const post = await getPostBySlug(slug);
const safeContent = DOMPurify.sanitize(post?.content || "");
---

<MainLayout title={post?.seo?.title || post?.title} description={post?.seo?.metaDesc}>
  <article class="gutenberg-article">
    <h1>{post?.title}</h1>
    <div class="gutenberg-content" set:html={safeContent} />
  </article>
</MainLayout>
```

7. Phase 5: Component Splits, Master Templates & Design Rules

7.1 Master Page Template Compositions

Standardize page layouts by assembling cohesive section components:

1. Service Detail Pages (src/pages/services/[service].astro):
   - <ServiceHero {...heroData} /> (Value proposition, badge, primary CTA)
   - <StartsSection {...startsData} /> (2-column spotlight + animated SVG stat ring)
   - <StatsBand {...statsData} /> (High-contrast animated KPI counters)
   - <ProcessSection {...processData} /> (4-step interactive accordion with synced graphic)
   - <PillarsSection {...pillarsData} /> (Numbered capability pillars)
   - <DeliverablesGrid {...deliverablesData} /> (3-column deliverables checklist)
   - <ReadyBand {...readyData} /> (Dark #1a1a1a CTA surface extending downward)
   - <ConsultSection {...consultData} /> (Animated SVG waves + React lead form island)
2. Case Study Pages (src/pages/work/[slug].astro):
   - <CaseStudyHero /> -> <CaseStudyStats /> -> <CaseStudyNarrative /> -> <LightboxModal /> -> <ConsultSection />

3. High-Converting Landing Pages (src/pages/landing/*.astro):
   - Wrapped in <LandingLayout /> (stripped-down navigation, focused conversion funnel)

7.2 Route-Scoped Wave & Background Geometry Integration

A signature visual technique across marketing sites is extending dark CTA surfaces directly behind animated wave ribbons:

- On /services routes:
  - ReadyBand.astro extends .ready-band__surface downward by the exact ribbon height:

```css
.ready-band__surface {
  padding-top: clamp(2rem, 5vw, 4rem);
  padding-bottom: clamp(55px, 8.5vw, 115px);
  margin-bottom: calc(-1 * clamp(55px, 8.5vw, 115px));
}
```

  - ConsultSection.astro checks Astro.url.pathname.includes("/services") and switches its wave theme to dark.
  - The SVG wave background remains transparent, and wave paths anchor to the bottom edge (M0,120 L1440,120 ...) with color: var(--color-cyan). The moving cyan waves rise seamlessly over the dark patterned background behind them.
- On standard pages outside /services:
  - waveTheme = "light" is active. The wave SVG masks the top edge in white (var(--color-bg)), providing a clean transition from white content sections into the cyan consultation area.

7.3 Motion & Layout Engineering Rules

1. Zero Text Occlusion: Floating satellite badges, background glow blobs, and decorative SVGs must never overlap text copy, headlines, or clickable buttons at any viewport width.
2. Anti-Clutter Media Composition:
   - 1 Centerpiece Illustration (public/assets/main-graphics/)
   - 1 Ambient Glow Backdrop (public/assets/decorations/)
   - At most 2 Opposite-Quadrant Satellites (top-left + bottom-right or bottom-left + top-right)
3. GSAP Motion Hygiene:
   - Wrap animations in gsap.matchMedia().add("(prefers-reduced-motion: no-preference)", () => { ... }).
   - Always apply clearProps: "transform,opacity" to entrance tweens so elements return to pure CSS layout after entering the viewport.

8. Phase 6: Gravity Forms React Island (LeadForm.jsx)

src/components/LeadForm.jsx is a self-contained React 19 component hydrated on demand:

- Inline: Inside ConsultSection.astro with client:visible (hydrates when scrolled into view).
- Modal: Inside the global popup dialog in MainLayout.astro (hydrates when any [data-open-modal] button is clicked).

Key Implementation Details:

- Unique IDs: Uses React useId() (${uid}-firstName, etc.) to prevent duplicate HTML id attributes when multiple forms exist in the DOM.
- Phone Auto-Masking: Formats phone digits automatically as (XXX) XXX-XXXX on keystroke and strips international prefixes on paste.
- GraphQL Mutation: Sends entries directly to https://<wp.domain.com>/graphql using the submitGfForm mutation.

JAVASCRIPT

```
// src/components/LeadForm.jsx
import React, { useState, useId } from "react";
import DOMPurify from "isomorphic-dompurify";
import "./LeadForm.css";

export const GF_FIELD_IDS = Object.freeze({
  COMPANY: 4,
  EMAIL: 5,
  PHONE: 6,
  SERVICES_NEEDED: 7,
  MESSAGE: 9,
  FIRST_NAME: 12,
  LAST_NAME: 13,
});

const SUBMIT_MUTATION = `
  mutation SubmitLead($input: SubmitGfFormInput!) {
    submitGfForm(input: $input) {
      confirmation { type message url }
      errors { id message }
    }
  }
`;

export default function LeadForm({ formId = 15, onSubmitted }) {
  const uid = useId();
  const [formData, setFormData] = useState({
    firstName: "",
    lastName: "",
    email: "",
    phone: "",
    company: "",
    services: [],
    message: "",
  });
  const [status, setStatus] = useState("idle"); // idle | submitting | success | error
  const [confirmationHtml, setConfirmationHtml] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();
    setStatus("submitting");

    const endpoint = (
      import.meta.env?.PUBLIC_WORDPRESS_GRAPHQL_URL ||
      "https://<wp.domain.com>/graphql"
    );

    const fieldValues = [
      { id: GF_FIELD_IDS.FIRST_NAME, value: formData.firstName },
      { id: GF_FIELD_IDS.LAST_NAME, value: formData.lastName },
      { id: GF_FIELD_IDS.EMAIL, emailValues: { value: formData.email } },
      { id: GF_FIELD_IDS.PHONE, value: formData.phone },
      { id: GF_FIELD_IDS.COMPANY, value: formData.company },
      { id: GF_FIELD_IDS.SERVICES_NEEDED, value: formData.services.join(", ") },
      { id: GF_FIELD_IDS.MESSAGE, value: formData.message },
    ];

    try {
      const res = await fetch(endpoint, {
        method: "POST",
```

```
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          query: SUBMIT_MUTATION,
          variables: { input: { id: formId, fieldValues } },
        }),
      });
      const data = await res.json();
      const confirmation = data.data?.submitGfForm?.confirmation;

      if (confirmation) {
        setStatus("success");
        setConfirmationHtml(DOMPurify.sanitize(confirmation.message || "Thank you! We will be in touch shor...
        if (onSubmitted) onSubmitted();
      } else {
        setStatus("error");
      }
    } catch (err) {
      console.error("Submission failed:", err);
      setStatus("error");
    }
  };

  // Form markup & JSX render implementation...
}
```

9. Phase 7: Vercel Deployment, DNS Cutover & Rebuild Webhooks

9.1 Vercel Project Setup

1. Push the <brand-frontend> repository to GitHub or GitLab (main branch).
2. Log in to Vercel -> Add New... -> Project -> Select the repository.
3. Configure Build Settings:
   - Framework Preset: Astro
   - Build Command: astro build (or npm run build)
   - Output Directory: dist
   - Install Command: npm install
   - Node.js Version: 22.x (verified in Project Settings -> General)
4. Add Environment Variables:
   - PUBLIC_WORDPRESS_API_URL = https://<wp.domain.com>/graphql
   - PUBLIC_WORDPRESS_GRAPHQL_URL = https://<wp.domain.com>/graphql
   - SITE_URL = https://<domain.com>
5. Click Deploy. Vercel will build the entire static site and provision an edge preview URL.

9.2 DNS Production Cutover Matrix

When launching production traffic, configure the domain's authoritative DNS zone:

Record Type	Host / Name	Target / Destination	TTL	Purpose
A	wp	<wp-server-ip>	300	Points wp.<domain.com> to Headless WordPress host
A	@	76.76.21.21	300	Points apex <domain.com> to Vercel Global Edge Network
CNAME	www	cname.vercel-dns.com	300	Directs www.<domain.com> to Vercel with auto-SSL

9.3 Configuring Vercel Automated Deploy Hooks

1. In Vercel, open Project Settings -> Git -> Deploy Hooks.
2. Create a new hook named WordPress CMS Content Update on the main branch.
3. Copy the unique hook URL (https://api.vercel.com/v1/integrations/deploy/prj_.../xxx).
4. Add the hook URL to your WordPress origin wp-config.php:

```php
define( '<PREFIX>_VERCEL_DEPLOY_HOOK', 'https://api.vercel.com/v1/integrations/deploy/...' );
```

Or paste it into WP Webhooks -> Send Data -> Post Updated / Post Created.

10. Phase 8: New Site Replication & Adaptability Checklist

Use this execution checklist whenever launching a new client or brand website on this stack:

[ ] 1. Project Bootstrap & Token Replacement:
- Clone or create repository with npm create astro@latest <brand-frontend>.
- Replace <domain.com>, <wp.domain.com>, and <PREFIX> placeholders across all config files.
- Update src/config/site.js with client phone numbers, notification emails, and social links.

[ ] 2. Design Tokens & Typography Customization (src/styles/global.css):
- Set primary brand color (--color-primary), accent cyan/teal (--color-cyan), dark canvas (--color-dark), and font families (--font-heading, --font-body).

[ ] 3. WordPress Origin Backend Setup:
- Install WPGraphQL, Gravity Forms, and WPGraphQL for Gravity Forms plugins on <wp.domain.com>.
- Deploy wp-content/mu-plugins/headless-cms-config.php with 500-item query limit and CORS origins.
- Whitelist /graphql in origin server WAF / Wordfence settings.
- Match GF_FIELD_IDS in LeadForm.jsx with the target Gravity Form.

[ ] 4. Content Modeling & Zero-Inline-CSS Compliance:
- Populate service and landing page models in src/data/**/*.js.
- Verify zero style="..." strings are present in data files; confirm layout coordinates use semantic placement tokens (placement: "top-left" | "top-right" | ...).

[ ] 5. Local Build & Performance Audit:
- Run npm run build locally to confirm all static routes compile with 0 errors.
- Inspect dist/sitemap-index.xml to verify automated search engine index generation.

[ ] 6. Edge Deployment & Form Testing:
- Link repository to Vercel and verify production build.
- Test LeadForm.jsx client-side validation, phone auto-masking, and live GraphQL mutation submission.
- Test mobile responsiveness across iOS and Android viewports, ensuring zero text occlusion.